

مستندات طراحی پروژه «فیلم‌بین»

اپلیکیشن مدیریت و دنبال کردن فیلم و سریال

درس برنامه‌سازی موبایل (۴۰۴۲۹) · مدرس: علی نجیمی

نام و نام خانوادگی: آرین اکبری

شماره دانشجویی: ۴۰۱۱۰۵۵۸۳

این پروژه چیست

فیلم‌بین یک اپلیکیشن اندرویدی است برای پیدا کردن فیلم و سریال، دنبال کردن قسمت‌های دیده‌شده، امتیاز دادن و نظر نوشتن. داده‌های آثار از IMDb می‌آید و فعالیت شخصی کاربر روی سرور خودمان می‌نشیند.

پروژه با **مدل پیشرفته** (بخش ۷ صورت پروژه) پیاده شده است. یعنی اپلیکیشن هیچ‌وقت مستقیم با IMDb حرف نمی‌زند؛ یک بک‌اند اختصاصی وسط ایستاده و کار گرفتن داده، یکدست‌کردنش، نگه‌داشتنش و کنترل دسترسی کاربرها را انجام می‌دهد.

چرا این مدل

سه دلیل عملی: اول اینکه کلید و جزئیات تماس با سرویس بیرونی از دست کاربر دور می‌ماند (بخش ۸.۳). دوم اینکه یک بار گرفتن داده برای همه کاربرها کافی است، پس هم سریع‌تر است هم فشار کمتری روی سرویس بیرونی می‌آورد (بخش ۷.۵). سوم اینکه اگر IMDb از دسترس خارج شود، برنامه با داده ذخیره‌شده سر پا می‌ماند.

معماری کلی

Flutter (اندروید) ← FastAPI ← IMDb GraphQL

مسیر یک درخواست از پایین به بالا: کاربر روی صفحه‌ای کاری می‌کند، صفحه از provider می‌خواهد، provider از مخزن، مخزن از ApiClient، و درخواست روی HTTP به بک‌اند می‌رسد. بک‌اند اول توی پایگاه داده خودش نگاه می‌کند؛ اگر داده تازه بود همان را برمی‌گرداند و اگر نبود از IMDb می‌گیرد، ذخیره می‌کند و بعد جواب می‌دهد.

چهار لایه اپلیکیشن و مسئولیتشان:

لایه	کارش
رابط کاربری	صفحه‌ها و ویجت‌ها. هیچ‌وقت مستقیم شبکه صدا نمی‌زند.
منطق برنامه	حالت، بارگذاری و خطا با Riverpod.
داده	مدل‌ها و مخزن‌ها. تنها جایی که JSON دیده می‌شود.
ذخیره‌سازی	sqlite برای آینه محلی، ذخیره امن برای توکن.

بک‌اند

بک‌اند با FastAPI نوشته شده و روی SQLAlchemy به صورت async با پایگاه داده کار می‌کند. ساختار پوشه‌ها:

پوشه	کارش
core/	تنظیمات، امنیت (JWT و bcrypt)، قالب خطا، محدودیت نرخ
db/	مدل‌های پایگاه داده و باز کردن اتصال
imdb/	کوئری‌های GraphQL، کلاینت مقاوم، نگاشت داده، لایه حافظه نهان
services/	منطق دامنه: پیشرفت تماشا، آمار، پیشنهادات، فعالیت‌ها
schemas/	قراردادهای ورودی و خروجی (Pydantic)
api/routes/	نقاط پایانی HTTP، دسته‌بندی‌شده بر اساس حوزه

گرفتن داده از IMDb

از نقطه پایانی GraphQL عمومی IMDb استفاده می‌کنیم – همان چیزی که خود سایت IMDb برای صفحه‌هایش صدا می‌زند. کلید نمی‌خواهد و همه چیز که صورت پروژه خواسته را دارد: جست‌وجوی پیشرفته، جزئیات اثر، فصل‌ها و قسمت‌ها، بازیگران و جدول‌های محبوبیت.

همه کوئری‌ها در یک فایل (imdb/queries.py) جمع شده‌اند تا اگر روزی سرویس اطلاعاتی عوض شود فقط همان یک فایل درگیر شود (بخش ۸.۶).

یک نکته عملی که موقع کار به آن خوردیم: لبه IMDb درخواستی را که سربرگ referer نداشته باشد رد می‌کند و ۴۰۳ می‌دهد، هرچقدر هم بقیه سربرگ‌ها شبیه مرورگر باشند. پس کلاینت origin و referer را هم می‌فرستد.

آینه محلی و حافظه نهان

هر پاسخی که از IMDb می‌گیریم در پایگاه داده خودمان می‌نشیند. نتیجه‌اش این است که درخواست تکراری به بیرون نمی‌رود، پاسخ‌ها سریع‌تر می‌شوند، و اگر IMDb قطع شود سرویس با داده ذخیره‌شده کار می‌کند. در پاسخ جست‌وجو یک پرچم stale هم هست که می‌گوید این نتیجه از IMDb آمده یا از آینه.

مدارشکن

اگر IMDb چند بار پشت سر هم خطا بدهد، کلاینت «مدار» را باز می‌کند و تا پایان یک بازه خنک‌شدن دیگر سراغش نمی‌رود. این کار جلوی تلنبار شدن درخواست‌های بی‌فایده و کند شدن کل سرویس را می‌گیرد؛ در این مدت بک‌اند از آینه محلی جواب می‌دهد.

احراز هویت

با JWT. توکن دسترسی کوتاه‌عمر است (۳۰ دقیقه) و توکن تازه‌سازی چرخشی، که به صورت درهم‌سازی‌شده در پایگاه داده نگه داشته می‌شود – نه خود توکن. اگر کاربر موقع ورود «مرا به

خاطر بسیار» را بزند نشست ۳۰ روزه می‌شود، وگرنه یک‌روزه (بخش ۵.۲).
دو سطح دسترسی داریم: کاربر عادی و مدیر سیستم. کارهای مدیریتی پشت یک وابستگی جدا هستند و کاربر عادی به آن‌ها نمی‌رسد (بخش ۷.۷).

قالب خطا

هر خطایی، از هر مسیری، یک شکل دارد تا اپلیکیشن فقط یک مسیر برای نمایش خطا داشته باشد:

```
{ "error": { "status": 404, "code": "TITLE_NOT_FOUND", "message": "...",  
  "detail": "tt0000000" } }
```

خطاهای اعتبارسنجی یک کلید fields هم دارند تا فرم بتواند پیام را دقیقاً زیر همان ورودی نشان دهد. پیام‌ها از قبل فارسی‌اند، پس اپلیکیشن می‌تواند مستقیم نشانشان بدهد و فقط برای واکنش برنامه‌ای به code نگاه کند.

پایگاه دادهٔ سمت سرور

جدول‌های اصلی: کاربران، نشست‌ها، آثار، فصل‌ها، قسمت‌ها، وضعیت تماشای کاربر، قسمت‌های دیده‌شده، امتیازها، نظرها، فهرست‌های شخصی، دنبال‌کردن کاربران، گزارش‌ها و پیام‌های گفت‌وگو. دو تصمیم که ارزش گفتن دارند:

یک وضعیت تماشا و علاقه‌مندی از هم جدا هستند. صورت پروژه «موردعلاقه» را هم در فهرست وضعیت‌ها آورده و هم به‌عنوان فهرست جداگانه (بخش‌های ۵.۹ و ۵.۱۶). در عمل کاربر می‌تواند هم‌زمان یک سریال را «در حال تماشا» و «موردعلاقه» داشته باشد، پس وضعیت یک enum پنج‌حالتی شد و علاقه‌مندی یک پرچم مستقل کنارش.

دو آثار به‌صورت تدریجی کامل می‌شوند. یک نتیجهٔ جست‌وجو فقط چند فیلد دارد؛ وقتی کاربر صفحهٔ اثر را باز کرد، همان ردیف با جزئیات کامل پر می‌شود. این‌طوری برای نشان دادن یک فهرست، بیست تا درخواست سنگین نمی‌زنیم.

اپلیکیشن

مدیریت حالت

با Riverpod. قاعده‌ها ساده‌اند: صفحه‌ها هیچ‌وقت مستقیم ApiClient را صدا نمی‌زنند، مخزن‌ها هیچ‌وقت ویجت نمی‌سازند، و همهٔ عددها از Formatters رد می‌شوند تا با ارقام فارسی نمایش داده شوند.

کار کردن بدون اینترنت

مسیر یک نوشتن آفلاین این‌طور است: مخزن اول روی دیسک می‌نویسد تا کاربر بلافاصله نتیجه را ببیند، بعد اگر شبکه نبود درخواست در یک «صندوق خروجی» می‌نشیند. دفعهٔ بعد که برنامه بالا می‌آید صف بخش می‌شود. عملیات خنثی‌پذیرند، پس اگر یک درخواست دو بار فرستاده شود امتیاز یا نظر تکراری ثبت نمی‌شود (بخش ۸.۴).

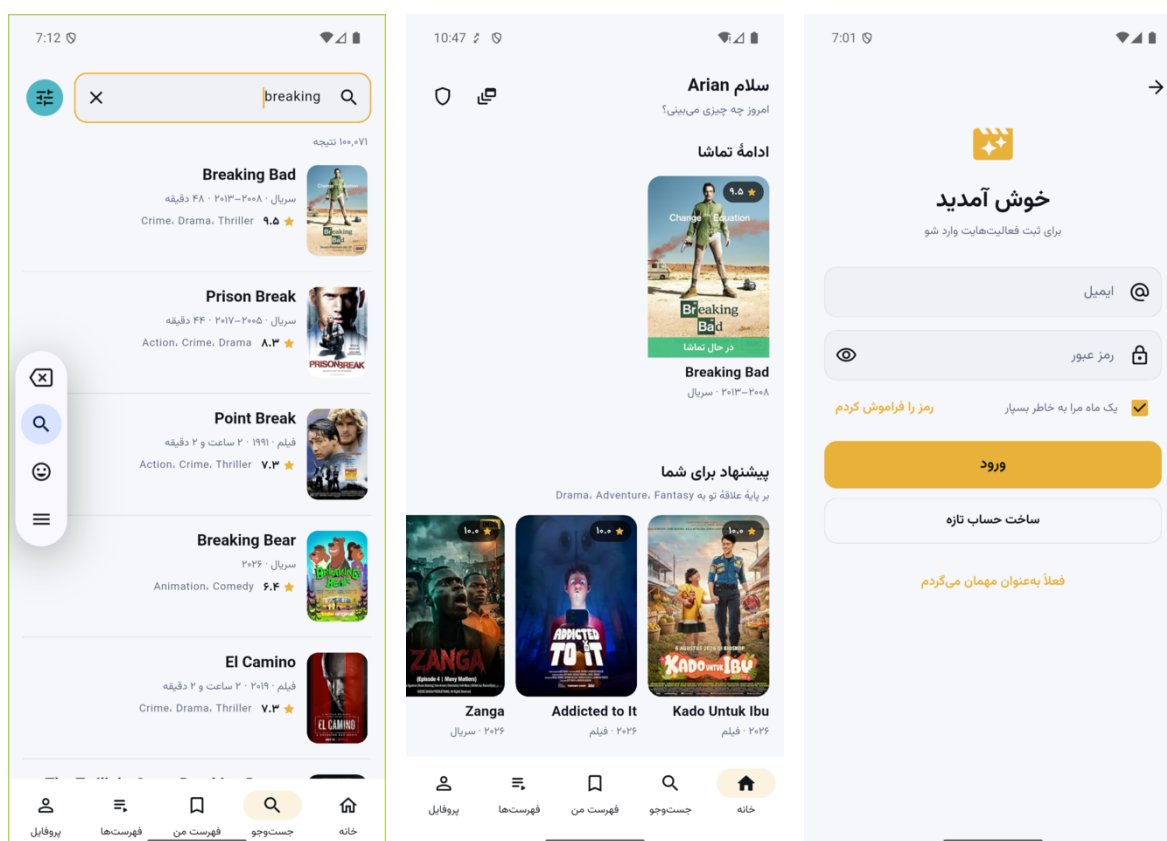
خواندن هم همین‌طور: آخرین چیزی که دیده شده در sqflite می‌ماند، پس برنامه بدون اتصال هم بالا

می‌آید و صفحه اصلی خالی نیست.

تصویرها

پوسترها در حافظه نهان روی دیسک می‌مانند و در اندازه‌ای که واقعاً نمایش داده می‌شوند درخواست می‌شوند، نه اندازه اصلی. این هم مصرف اینترنت را پایین می‌آورد هم حافظه را (بخش ۸.۸).

صفحه‌ها و کار با برنامه



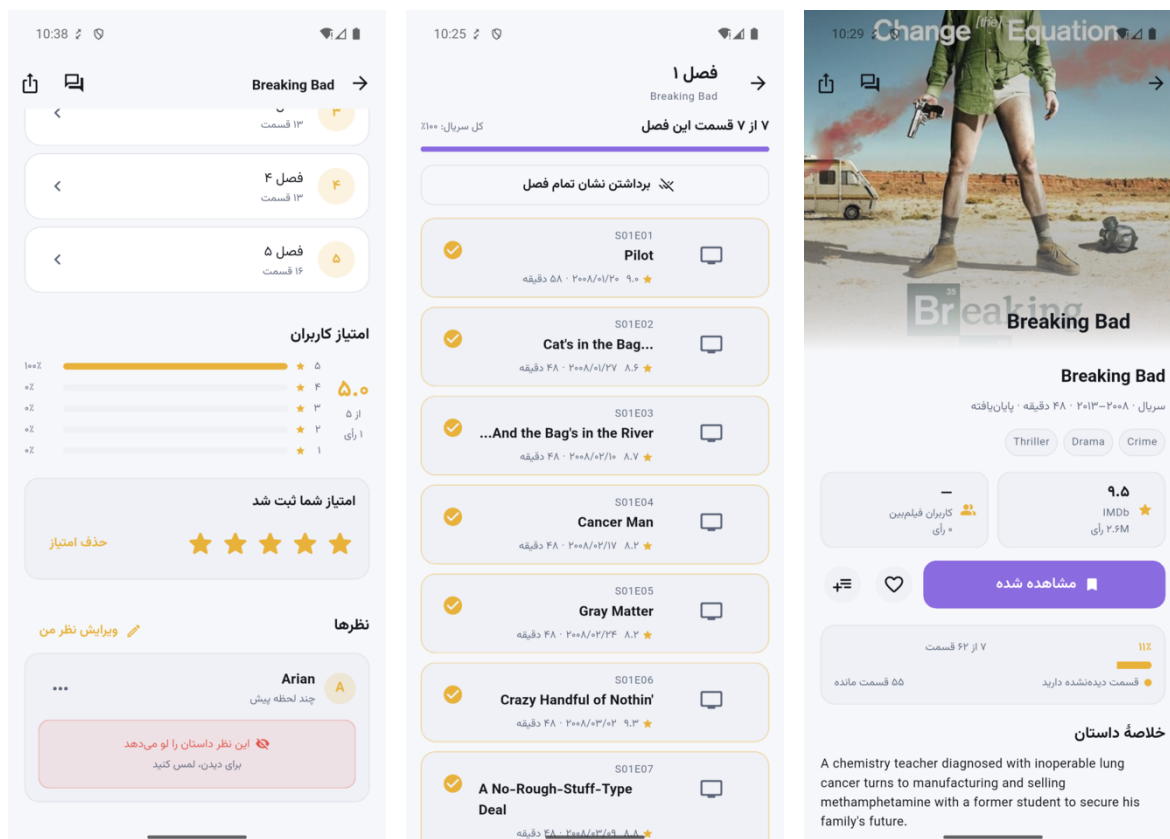
ورود · صفحه اصلی · جست‌وجو

ورود و ثبت‌نام. ثبت‌نام با نام، نام کاربری، ایمیل و رمز عبور انجام می‌شود؛ تصویر پروفایل و توضیح کوتاه اختیاری‌اند. ایمیل و نام کاربری تکراری همان‌جا رد می‌شوند. اگر رمز یادتان برود، از «فراموشی رمز» می‌شود آن را بازیابی کرد.

کاربر مهمان بدون ثبت‌نام هم می‌تواند جست‌وجو کند، صفحه آثار را ببیند و آثار محبوب و فصل‌ها و قسمت‌ها را مرور کند. برای ثبت هر فعالیت (وضعیت، امتیاز، نظر، فهرست) ورود لازم است.

صفحه اصلی. ردیف‌های فیلم‌های محبوب، سریال‌های محبوب، آثار جدید، آثار با امتیاز بالا و سریال‌های برتر. برای کاربری که فعالیت داشته، یک ردیف «پیشنهاد برای شما» هم می‌آید که بر پایه ژانرهای واقعی خودش ساخته می‌شود. تا وقتی سیگنالی نباشد این ردیف اصلاً نشان داده نمی‌شود تا همان ردیف محبوب‌ها دوباره تکرار نشود.

جست‌وجو. دست‌کم بر اساس نام اثر کار می‌کند و پالایه‌های ژانر، سال، نوع اثر و نام بازیگر یا کارگردان هم دارد. تایپ که می‌کنید درخواست بلافاصله نمی‌رود؛ کمی صبر می‌کند تا دست کاربر بایستد و بعد یک درخواست بفرستد.

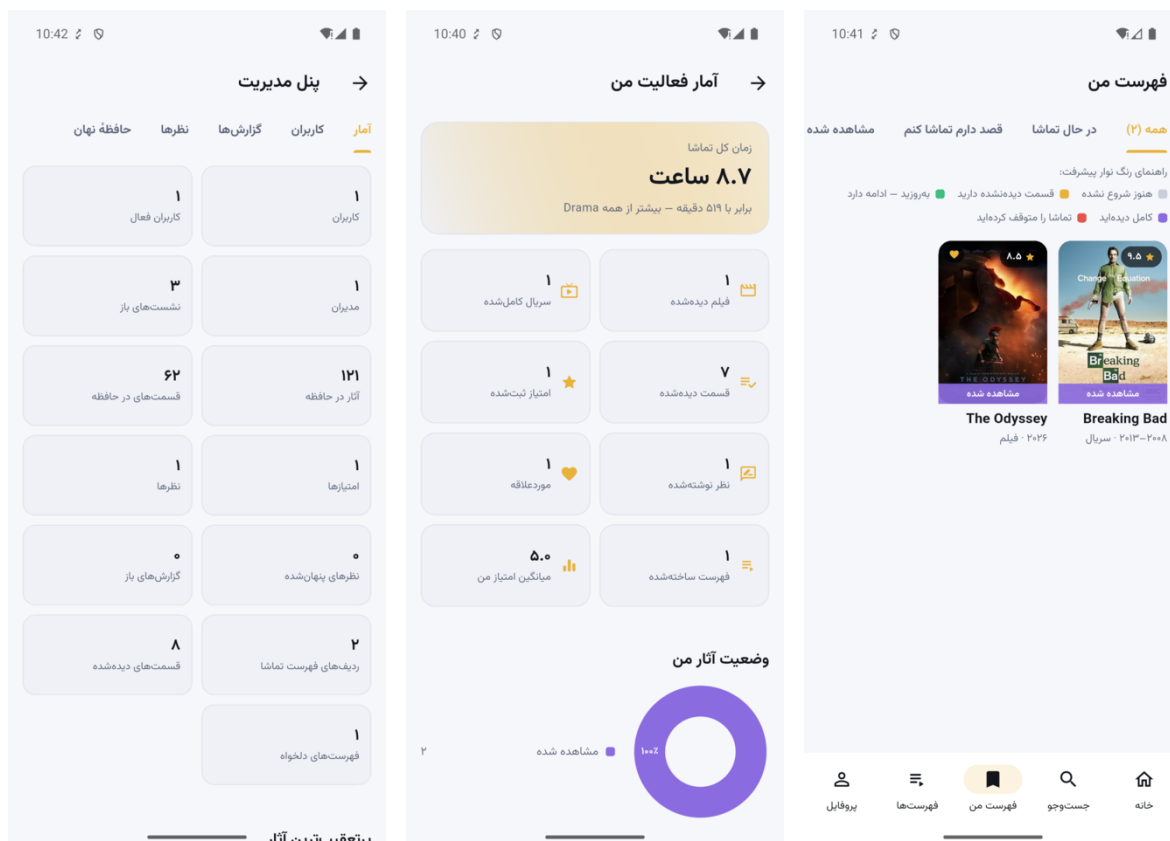


صفحه اثر · قسمت‌ها · امتیاز و نظر

صفحه اثر. برای فیلم: نام، نام اصلی، پوستر، خلاصه داستان، سال، مدت زمان، ژانر، کشور سازنده، کارگردان، بازیگران، امتیاز IMDb و امتیاز کاربران خود برنامه. برای سریال به جای مدت زمان، سال شروع و پایان پخش، وضعیت پخش، تعداد فصل‌ها و تعداد قسمت‌ها می‌آید.

فصل‌ها و قسمت‌ها. هر قسمت شماره فصل، شماره قسمت، عنوان، تاریخ پخش، مدت زمان، خلاصه و وضعیت دیده شدن دارد. می‌شود یک قسمت را تیک زد یا کل یک فصل را یکجا علامت زد. تعداد دیده شده و باقی مانده همان‌جا حساب می‌شود.

امتیاز و نظر. امتیاز از یک تا پنج ستاره است و توزیع درصدی ستاره‌ها زیرش نشان داده می‌شود. امتیاز قبلی قابل ویرایش است. نظر شامل متن، نام و تصویر کاربر، تاریخ ثبت و وضعیت اسپویل است؛ نظر اسپویل‌دار اول پوشانده می‌شود و فقط با زدن روی آن باز می‌شود.



فهرست تماشا · آمار فعالیت · پنل مدیر

فهرست تماشا و فهرست‌های شخصی. پنج وضعیت داریم: قصد دارم تماشا کنم، در حال تماشا، مشاهده شده، متوقف شده و رها شده. «موردعلاقه» کنار این‌ها یک پرچم جداساز. علاوه بر این‌ها کاربر می‌تواند فهرست دلخواه بسازد («بهترین فیلم‌های اکشن»، «فیلم‌هایی که باید ببینم») و آثار را به آن اضافه یا از آن حذف کند.

آمار فعالیت. تعداد فیلم‌های دیده شده، سریال‌های دیده شده، قسمت‌های دیده شده، مجموع زمان تقریبی تماشا، ژانر موردعلاقه و میانگین امتیازهای ثبت شده.

پنل مدیر. مدیر سیستم می‌تواند کاربرها را ببیند و مدیریت کند، نظریات نامناسب را حذف کند، گزارش‌های کاربرها را بررسی کند و آمار کلی سامانه را ببیند.

نوار پیشرفت و رنگ‌بندی‌اش

درصد پیشرفت تماشای سریال هم به صورت عدد و هم به صورت نوار زیر پوستر نشان داده می‌شود که عرضش به همان نسبت پر می‌شود. رنگ نوار روی سرور حساب می‌شود تا همه کلاینت‌ها یک چیز نشان بدهند:

رنگ	یعنی
بی‌رنگ	هنوز هیچ قسمتی دیده نشده
زرد	قسمت دیده‌نشده باقی مانده
سبز	همه را دیده‌ای، ولی سریال ادامه دارد و قسمت جدید می‌آید
بنفش	همه را دیده‌ای و سریال تمام شده
قرمز	وسط کار متوقف کرده‌ای و کامل ندیده‌ای

پوشش خواسته‌های صورت پروژه

بخش ۵ – نیازمندی‌های کاربردی

بند	کجا پیاده شده
۵.۱ ثبت‌نام	routes/auth.py + register_screen.dart
۵.۲ ورود و خروج	login_screen.dart، نشست ۳۰ روزه
۵.۳ بازیابی رمز	forgot_password_screen.dart
۵.۴ مدیریت پروفایل	screens/profile/
۵.۵ جست‌وجو	search_screen.dart
۵.۶ جزئیات فیلم	title_detail_screen.dart
۵.۷ جزئیات سریال	همان صفحه، حالت سریال
۵.۸ فصل‌ها و قسمت‌ها	season_screen.dart
۵.۹ وضعیت تماشا	routes/tracking.py + WatchStatus
۵.۱۰ علامت‌گذاری قسمت	EpisodeTile
۵.۱۱ نوار پیشرفت	progress_bar.dart + services/progress.py
۵.۱۲ فهرست تماشا	watchlist_screen.dart
۵.۱۳ امتیازدهی	services/titles.py + rating_widgets.dart
۵.۱۴ ثبت نظر	ReviewComposer

بند	کجا پیاده شده
۵.۱۵ اسپویل	ReviewTile
۵.۱۶ علاقه‌مندی‌ها	پرچم is_favorite
۵.۱۷ فهرست‌های شخصی	screens/lists/
۵.۱۸ آثار محبوب	titles/discover + home_screen.dart
۵.۱۹ آمار کاربر	services/stats.py + stats_screen.dart
۵.۲۰ مدیریت خطا	ErrorView + ApiException

بخش ۷ – بک‌اند اختصاصی

طراحی بک‌اند با FastAPI، API اختصاصی برای همهٔ عملیات‌های خواسته‌شده، واسط IMDb با یکدست‌سازی خروجی، پایگاه دادهٔ سمت سرور، حافظهٔ نهان و آینهٔ آثار، احراز هویت JWT، دو سطح دسترسی، اعتبارسنجی سمت سرور، مدیریت خطا با قالب یکسان، و مستندات OpenAPI فارسی روی مسیر docs.

بخش ۸ – نیازمندی‌های غیرکاربردی

بند	چه شد
۸.۱ کارایی	حافظهٔ نهان دو طرف، جست‌وجوی تأخیردار، بارگذاری تدریجی، اسکلت بارگذاری
۸.۲ کاربردپذیری	فارسی و راست‌به‌چپ، ارقام فارسی، اعتبارسنجی فرم قبل از ارسال
۸.۳ امنیت	bcrypt، JWT کوتاه‌عمر، توکن درهم‌سازی‌شده، محدودیت نرخ، HTTPS و سنجاق گواهی
۸.۴ اتکاپذیری	صندوق خروجی آفلاین، آینهٔ محلی، مدارشکن، عملیات خنثی‌پذیر
۸.۵ سازگاری	تم تیره و روشن، چیدمان واکنش‌گرا، محدود کردن بزرگ‌نمایی متن
۸.۶ نگهداری	لایه‌بندی روشن، تحلیلگر بدون هشدار، ۲۲۱ آزمون
۸.۷ مقیاس‌پذیری	سرویس بی‌حالت، آمادهٔ PostgreSQL از راه DATABASE_URL
۸.۸ مصرف منابع	تصویر در اندازهٔ نمایش، حافظهٔ نهان پویا، حالت کم‌مصرف

امنیت

رمز عبور با bcrypt و نمک جداگانه برای هر رمز ذخیره می‌شود؛ هیچ‌وقت متن ساده در پایگاه داده نمی‌نشیند. توکن تازه‌سازی هم خودش ذخیره نمی‌شود، فقط درهم‌سازی‌اش. سمت اپلیکیشن توکن در حافظه امن سیستم‌عامل می‌ماند نه در فایل ساده.

برای HTTPS یک CA خصوصی و گواهی سرور ساخته می‌شود و اثر انگشت SHA-256 همان گواهی موقع اجرا به اپلیکیشن داده می‌شود تا فقط همین سرور را قبول کند (سنجاق گواهی). ورودی‌ها هم سمت اپلیکیشن و هم سمت سرور اعتبارسنجی می‌شوند، و مسیرهای حساس محدودیت نرخ دارند.

آزمون‌ها

تعداد	چه چیزی
۱۱۵	بک‌اند، بدون شبکه
۱۰۶	اپلیکیشن (مدل، شبکه، مخزن، ویجت، صفحه)
۳	قراردادی روی IMDb واقعی

آزمون‌های صفحه‌ها مخزن‌ها را جایگزین می‌کنند، پس شبکه‌ای در کار نیست و هر آزمون همان چیزی را می‌سنجد که کاربر می‌بیند. flutter analyze هم بدون هیچ هشدار تمام می‌شود.

راه‌اندازی

بک‌اند

```
cd backend
python3 -m venv .venv
.venv/bin/pip install -r requirements-dev.txt
.venv/bin/uvicorn app.main:app --host 0.0.0.0 --port 8000
```

سلامت سرویس روی مسیر api/v1/health و مستندات تعاملی روی مسیر docs بالا می‌آید.

اپلیکیشن

```
cd app
flutter pub get
flutter run -d <device> \
  --dart-define=API_BASE_URL=http://10.0.2.2:8000/api/v1
```

10.0.2.2 یعنی «همین کامپیوتر» از دید شبیه‌ساز اندروید. روی دستگاه واقعی به‌جایش IP کامپیوتر را بگذارید.